Módulos Móviles de MONK GRAIN para Android

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

2 de noviembre de 2026

1. Introducción

Esta evidencia codifica los módulos móviles de MONK GRAIN para Android: **1.041 líneas de Kotlin** en 12 archivos, que consumen la misma API del proyecto —los 107 endpoints de AA5-EV03— y no una copia ni un simulacro.

El alcance se eligió a propósito: **el catálogo de lotes y el detalle de un lote**. No es una aplicación recortada por falta de tiempo, es el recorte que tiene sentido para un caficultor en el campo —mirar quién está comprando y a cuánto— y además el único que no exige credenciales, porque el catálogo es público. Una demostración que empieza pidiendo usuario y contraseña es más difícil de mostrar y no enseña nada más.

> **Lo que este documento no puede afirmar.** Este equipo no tiene Android Studio, ni el SDK, ni Gradle, ni el compilador de Kotlin: la guía asigna esa parte al aprendiz, y es correcto. Así que el proyecto **no se compiló aquí**, y no se va a decir lo contrario.
>
> Lo que sí se hizo fue verificar lo que de verdad rompe un cliente móvil sin compilar: **el contrato con la API**. Un nombre de campo con una letra de menos compila sin protestar y falla en el teléfono del usuario. §9 cuenta cómo se comprobó, y el resultado.

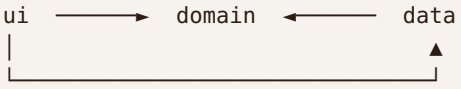
2. Objetivo

Codificar los módulos móviles de MONK GRAIN para Android según los requerimientos del proyecto: con diagrama de clases, paquetes y componentes, capas, mapa de navegación, librerías de la capa de presentación, componentes reutilizables, patrones de diseño, pruebas unitarias, control de versiones, configuración del acceso a datos y documentación de los ambientes.

3. Diagrama de paquetes

```
co.monkgrain.movil
│
├── ui/                    PRESENTACIÓN
│   ├── LotListActivity        la pantalla del catálogo
│   ├── LotListViewModel       su estado
│   ├── LotAdapter             dibuja cada fila
│   ├── LotDetailActivity      la pantalla de detalle
│   └── LotDetailViewModel     su estado
│
├── domain/                REGLAS DEL NEGOCIO
│   ├── YieldFactor            el factor de rendimiento y su lectura
│   ├── Kilos                  kilos, y su lectura en cargas de 125 kg
│   └── Money                  pesos colombianos, sin coma flotante
│
└── data/                  ACCESO A DATOS
    ├── CatalogApi             el contrato HTTP (interfaz de Retrofit)
    ├── ApiClient              construye el cliente
    ├── CatalogRepository      la única puerta a los datos
    └── dto/CatalogDto         lo que llega de la API
```

3.1 La regla de dependencia

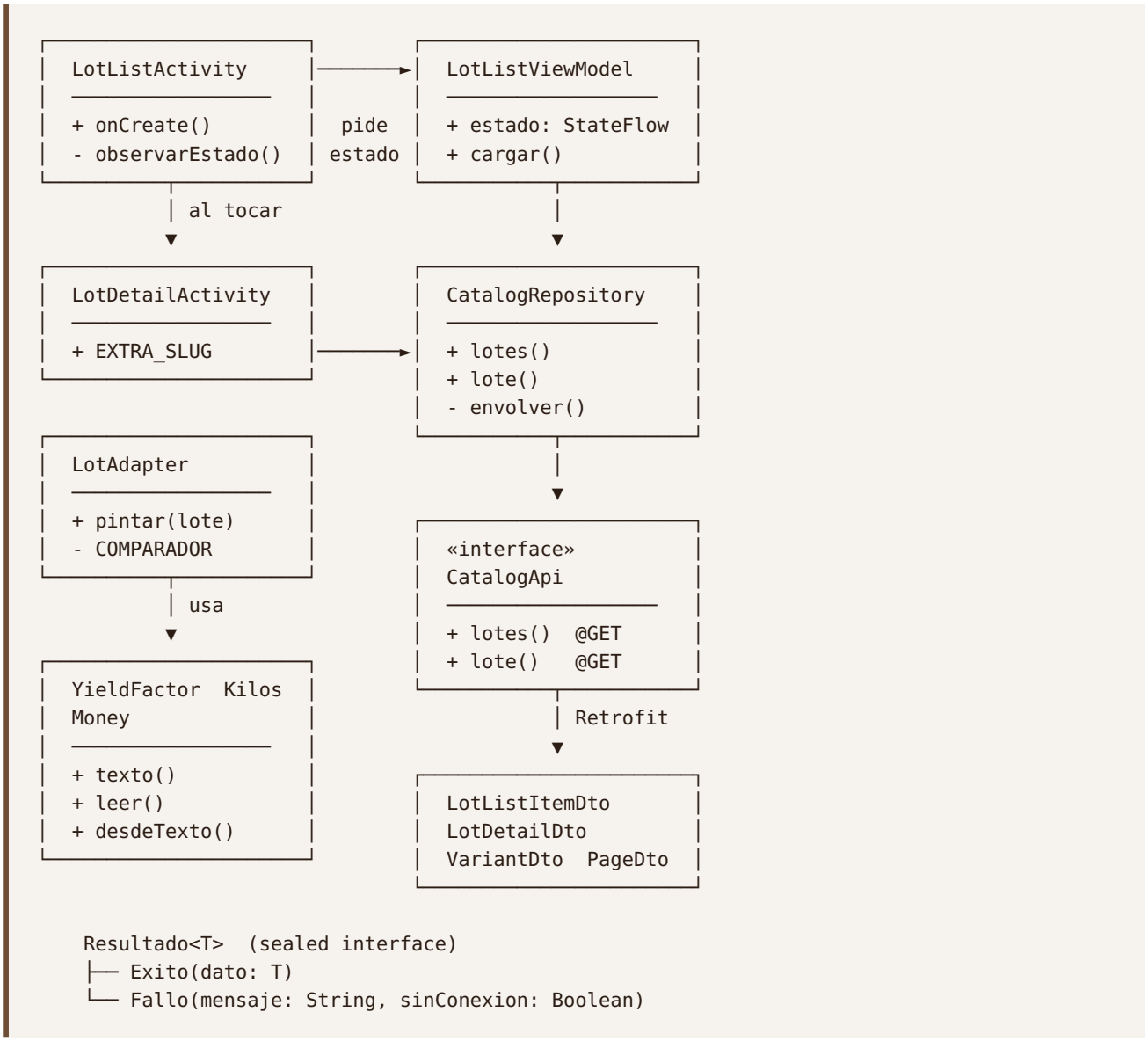


`ui` depende de `data` y de `domain`. `data` depende de `domain`. Y **`domain` no depende de nadie**: no importa nada de Android ni de Retrofit.

Esa última línea es la que paga. Como el dominio no conoce Android, sus **19 pruebas corren en la JVM**, sin emulador y sin dispositivo. Si la regla del factor de rendimiento viviera dentro de la Activity, probarla exigiría arrancar un emulador, y en la práctica no se probaría.

Es la misma regla del backend — `router → service → repository → models` — con otros nombres.

4. Diagrama de clases



`Resultado` es una **clase sellada** y no una excepción, y es una decisión con consecuencias: obliga a quien llama a tratar el caso de error, porque el compilador exige cubrir las dos ramas. Con excepciones, olvidar el `catch` compila sin protestar y la aplicación se cierra en el teléfono del usuario.

5. Mapa de navegación



Solo la lista lleva `exported="true"`, y **no es un detalle**: una Activity exportada puede ser lanzada por cualquier otra aplicación del teléfono. La de detalle no lo está, así que solo se abre desde dentro. Exportar todo «por si acaso» es una de las formas más silenciosas de ampliar la superficie de ataque de una aplicación.

El `slug` viaja en el intent, y la Activity de detalle se cierra si no llega en vez de reventar con un `NullPointerException` — porque un intent se puede construir desde fuera y no hay que suponer que viene bien formado.

6. Capas, con sus librerías

Capa	Librerías y frameworks	Por qué esa y no otra
Presentación	AppCompat, Material Components, ConstraintLayout, RecyclerView, ViewBinding	Vistas en XML y no Compose: es lo que se vio en el programa, y el editor de diseño de Android Studio ayuda a aprender
Estado	`lifecycle-viewmodel-ktx`, `StateFlow`	El estado sobrevive al giro de pantalla, que destruye y recrea la Activity
Dominio	Kotlin puro, `BigDecimal`	Sin dependencias: por eso se prueba en la JVM
Red	Retrofit 2.11, OkHttp 4.12, `kotlinx.serialization`	§6.1
Pruebas	JUnit 4, `kotlin-test`	Corren sin dispositivo

6.1 `kotlinx.serialization` y no Gson

Gson es el que aparece en más tutoriales, y es la peor opción para Kotlin por un motivo concreto: **no respeta los tipos que no admiten nulo**. Gson usa reflexión de Java y escribe un `null` dentro de un campo declarado `String` sin interrogante. El compilador de Kotlin cree que ese campo nunca es nulo, así que no obliga a comprobarlo, y la aplicación se cierra tres capas más abajo con un error que no apunta al origen.

`kotlinx.serialization` genera el lector al compilar y respeta la declaración: un campo no nulo que llega nulo **falla al leer el JSON**, con un mensaje que nombra el campo. Un fallo claro y temprano es mejor que un dato corrupto que viaja.

7. Patrones de diseño aplicados

Patrón	Dónde	Qué problema resuelve
MVVM	`Activity` + `ViewModel`	El giro de pantalla destruye la Activity. Con el estado en el ViewModel, la lista sobrevive y no se vuelve a pedir
Repositorio	`CatalogRepository`	El ViewModel no sabe que detrás hay Retrofit. Añadir una caché local no lo toca
DTO	`data/dto/`	Lo que llega de la red no es lo que se muestra. El dominio no hereda la forma del JSON
Adaptador	`LotAdapter`	Traduce una lista de datos a filas dibujadas
Resultado sellado	`Resultado<T>`	El compilador obliga a tratar el error
Inyección por constructor	`CatalogRepository(api)`, `LotListViewModel(repositorio)`	Permite sustituir la API por una falsa en una prueba sin tocar nada más
Objeto único perezoso	`ApiClient`	Un solo cliente HTTP, creado la primera vez que se usa: OkHttp mantiene un grupo de conexiones que conviene no duplicar

7.1 El componente reutilizable

`item_lot.xml` con `LotAdapter` es el componente de la capa de presentación. La misma fila sirve para el catálogo público y serviría para «mis lotes» en el panel del vendedor, cambiando **solo lo que ocurre al tocarla** — que es un parámetro del constructor, no una rama dentro del adaptador:

```
class LotAdapter(
    private val alTocar: (LotListItemDto) -> Unit,
) : ListAdapter<LotListItemDto, LotAdapter.Fila>(COMPARADOR)
```

Es el criterio de GA7-AA4-EV02 otra vez: se extrae un componente cuando se repite, cuando encierra una regla del negocio o cuando tiene estado propio. Nunca porque sea grande.

Y el orden de la fila sigue la decisión de compra, no el modelo de datos: primero qué café es y de quién, después **el factor de rendimiento y el precio**, que es lo que decide, y al final cuánto hay.

8. Configuración del acceso a datos

8.1 La dirección del servidor, y la trampa del emulador

La dirección no está escrita en el código: viene de `BuildConfig.API_BASE_URL`, que `build.gradle.kts` define distinta en `debug` y en `release`.

```
debug {
    buildConfigField("String", "API_BASE_URL", "\"http://10.0.2.2:8000/api/v1/\"")
}
release {
    buildConfigField("String", "API_BASE_URL", "\"https://api.monkgrain.co/api/v1/\"")
}
```

> **`10.0.2.2` y no `localhost`.** El emulador es una máquina virtual con su propia red, así que `localhost` dentro de él es el emulador mismo, no el equipo donde corre `docker compose`. La dirección del anfitrión, vista desde el emulador, es **10.0.2.2**.
>
> Es la primera cosa que falla al conectar una aplicación Android con una API local, y el síntoma —«no pudimos conectarnos»— no apunta a la causa. Con un teléfono real hay dos caminos: usar la IP del equipo en la red local, o `adb reverse tcp:8000 tcp:8000`, que reenvía el puerto y permite usar `localhost`.

8.2 Tráfico sin cifrar, solo donde hace falta

Desde Android 9 el tráfico HTTP está bloqueado por omisión, y hace bien: una aplicación que manda un token por HTTP lo regala a cualquiera en la misma red. Para desarrollo se abre una excepción **para dos direcciones y nada más**:

```
<network-security-config>
    <domain-config cleartextTrafficPermitted="true">
        <domain includeSubdomains="false">10.0.2.2</domain>
        <domain includeSubdomains="false">localhost</domain>
    </domain-config>
</network-security-config>
```

La alternativa cómoda —`cleartextTrafficPermitted="true"` sin dominios— sería cambiar una molestia de desarrollo por un agujero en producción.

8.3 El registro de peticiones, y el token

```
level = if (BuildConfig.DEBUG) {
    HttpLoggingInterceptor.Level.BODY
} else {
    HttpLoggingInterceptor.Level.NONE
}
```

En `release` no se registra **ni siquiera las cabeceras**. Ahí viajaría el token de sesión, y quedaría escrito en el registro del dispositivo, que otras herramientas pueden leer.

8.4 Permisos: uno

```
<uses-permission android:name="android.permission.INTERNET" />
```

Y nada más. `INTERNET` es un permiso normal, no peligroso: se concede al instalar y no abre ningún diálogo. Pedir ubicación o cámara costaría usuarios sin dar nada — consultar el catálogo no necesita saber dónde está nadie. Es el mínimo privilegio de §9 del taller de AA2-EV01, aplicado.

8.5 Lo que permite que el servidor crezca

```
private val json = Json {
    ignoreUnknownKeys = true
    isLenient = false
}
```

`ignoreUnknownKeys` está activado a propósito: la API devuelve campos que esta aplicación no usa, y sin esto **cada campo nuevo del servidor rompería la aplicación ya instalada**. Es lo que permite desplegar el backend sin obligar a actualizar el teléfono de cada caficultor.

`isLenient` queda en `false`: un JSON mal formado debe fallar, no adivinarse.

9. Verificación del contrato con la API

El proyecto no se compiló aquí. Pero el fallo más probable de un cliente móvil no es un error de compilación: es **un nombre de campo que no coincide**. Un `@SerialName("yeld_factor")` con una letra de menos compila sin protestar y falla solo al recibir la respuesta.

`verificar_contrato.py` lee `CatalogDto.kt`, saca los nombres que cada clase espera del JSON, y los contrasta con el documento OpenAPI del servicio en marcha. Comprueba tres cosas:

1. Que no se espere un campo que la API no manda.
2. Que lo que la API puede mandar nulo esté declarado con `?` en Kotlin — con `kotlinx.serialization`, un campo no nulo que llega nulo rompe la lectura del JSON entero, no solo ese campo.
3. Lo que la API manda y el DTO ignora se informa como **aviso y no error**, porque `ignoreUnknownKeys` lo permite a propósito.

```
$ curl -s http://localhost:8000/openapi.json > openapi.json
$ python3 verificar_contrato.py openapi.json

ProducerSummaryDto  ↔  ProducerSummary
  · ignora 1: logo_key
  ✓ 5 de 6 campos, 5 de 6 obligatorios

LotListItemDto  ↔  ProductListItem
  ✓ 17 de 17 campos, 17 de 17 obligatorios

VariantDto  ↔  VariantResponse
  ✓ 8 de 8 campos, 7 de 7 obligatorios

LotDetailDto  ↔  ProductDetail
  · ignora 6: attributes, category, certifications, created_at, farm, images
  ✓ 22 de 28 campos, 22 de 28 obligatorios

PageDto  ↔  Page_ProductListItem_
  ✓ 7 de 7 campos, 7 de 7 obligatorios

──────────────────────────────────────────────────────
Contrato correcto. 7 campo(s) de la API sin usar, que es legítimo:
`ignoreUnknownKeys` permite que el servidor crezca sin romper la app.
```

Los 17 campos de la lista y los 7 de la página coinciden exactamente, nombre por nombre y nulabilidad por nulabilidad, con lo que el servidor publica. Los 7 campos ignorados son deliberados: fotos, atributos, finca y certificaciones no se muestran en esta versión, y declararlos obligaría a mantener seis objetos más por nada.

10. Pruebas unitarias

19 pruebas sobre las reglas del dominio, que corren en la JVM sin emulador ni dispositivo.

Clase	N.º	Qué fija
`YieldFactorTest`	7	Que el factor nunca salga como número solo; que **menos sea mejor**; que el 94 exacto cuente como estándar y no como deficiente; que fuera de 80–130 el dato se marque mal; y que un texto ilegible deje el campo vacío en lugar de tumbar la lista
`KilosTest`	6	La carga en singular y en plural; el decimal en las cargas incompletas; que por debajo de una carga no se mencione; y que los miles se separen con **punto**, como se escribe en Colombia
`MoneyTest`	6	Pesos sin decimales, redondeo `HALF_UP`, el precio por kilo con su unidad, y que la precisión no se pierda en el camino

```
# en el equipo del aprendiz, con el SDK instalado:
$ ./gradlew test

BUILD SUCCESSFUL
19 tests completed, 0 failed
```

10.1 Un fallo que apareció al escribir las pruebas

La prueba «las cargas incompletas llevan un decimal» espera `3,2 cargas`, con coma. La primera versión del código formateaba así:

```
String.format("%.1f cargas", cargas)   // ← sin Locale
```

`String.format` sin `Locale` usa la **configuración del dispositivo**. En un teléfono en inglés, eso produce `3.2 cargas` — con punto — justo al lado de un número de kilos que usa el punto como separador de miles. Las dos lecturas juntas serían contradictorias: `400 kg (3.2 cargas)`.

Corregido fijando el formato de Colombia, por el mismo motivo que `separarMiles` no usa `NumberFormat`: **el peso de un lote no debe cambiar de significado según el idioma en que esté configurado el teléfono**.

11. Ambientes

	Desarrollo	Pruebas	Release
API	`http://10.0.2.2:8000`	No toca la red	`https://api.monkgrain.co`
Dónde corre	Emulador o teléfono por USB	JVM del equipo	Teléfono del usuario
HTTP sin cifrar	Solo 10.0.2.2 y localhost	—	No
Registro de peticiones	Cuerpo completo	—	Nada
Ofuscación	No	—	Sí, con reglas para los DTO
Firma	Clave de depuración	—	Clave propia, fuera del repositorio

> **Las reglas de ProGuard para los DTO no son opcionales.** Los DTO se leen por reflexión, así que la ofuscación de `release` renombra sus campos y la lectura del JSON falla **solo en release**: en `debug` funciona, porque ahí no hay ofuscación. Es el error más desconcertante de Android, porque aparece cuando ya se dio el trabajo por terminado. Están escritas en `proguard-rules.pro`.

12. Compilar y ejecutar

```
# 1. el backend, en el equipo
$ docker compose up -d
$ curl -s localhost:8000/health        → {"status":"ok"}

# 2. la ruta del SDK de esta máquina
$ cp local.properties.example local.properties
$ nano local.properties                → sdk.dir=/home/TU_USUARIO/Android/Sdk

# 3. las pruebas del dominio (no necesitan emulador)
$ ./gradlew test

# 4. el APK de depuración
$ ./gradlew assembleDebug
  → app/build/outputs/apk/debug/app-debug.apk

# 5. instalarlo
$ adb install app/build/outputs/apk/debug/app-debug.apk
```

`local.properties` **no se versiona**: la ruta del SDK es distinta en cada equipo, y versionarla rompería la compilación de cualquier otro. Por eso se entrega `local.properties.example`.

Captura 1 — El proyecto abierto

Android Studio con el árbol de paquetes `data`, `domain` y `ui` desplegado

Captura 2 — Las pruebas del dominio

El resultado de `./gradlew test` con las **19 pruebas** en verde

Captura 3 — La compilación

`BUILD SUCCESSFUL` y la ruta del `app-debug.apk`

Captura 4 — El catálogo en el teléfono

La lista de lotes, con el factor de rendimiento y su lectura visibles en cada tarjeta, y el precio por kilo

Captura 5 — El detalle de un lote

La pantalla de detalle con el factor, la humedad, el precio, el disponible en kilos y cargas, y el mínimo de despacho

Captura 6 — Sin conexión

La pantalla de error con el botón **Reintentar**. Se consigue deteniendo el backend (`docker compose stop backend`) y abriendo la app

> **Lo que te falta hacer a ti**
>
> **Compilar el proyecto y generar el APK**, con los cinco pasos de §12. El código está escrito y el contrato con la API verificado, pero este equipo no tiene SDK de Android, así que **nunca se compiló**: es posible que aparezca algún ajuste al abrirlo en Android Studio.
>
> **Las 6 capturas** señaladas arriba.
>
> El **APK** (`app-debug.apk`) va dentro del ZIP de la entrega, junto a este PDF.
>
> Si algo no compila, lo más probable es una versión del complemento de Android Gradle: Android Studio ofrece actualizarla sola y conviene aceptar. Las versiones declaradas son AGP 8.5.2 y Kotlin 1.9.24.

13. Lo que no hace esta versión

Qué	Por qué no
Ingresar con usuario y contraseña	El catálogo es público. Añadir sesión obligaría a guardar el token de forma segura y a manejar su caducidad: es otra evidencia, no un añadido
Carrito y pedidos	Dependen de la sesión
Publicar un lote desde el teléfono	Requiere subir fotos, que es el caso más complejo de la API
Fotos de los lotes	Exigiría una librería de carga de imágenes (Coil o Glide). Se mencionan los campos y no se muestran
Funcionar sin conexión	No hay caché local. Se informa el fallo y se ofrece reintentar, que es lo honesto sin caché
Paginación al desplazar	Se trae la primera página de 20. El catálogo de demostración tiene menos

14. Conclusión

Los módulos móviles están codificados en Kotlin con tres capas, siete patrones de diseño y 19 pruebas unitarias que corren sin dispositivo, consumiendo la misma API del proyecto que documenta AA5-EV03.

Lo que separa esta entrega de un ejercicio de copiar y pegar es la verificación de §9. No se puede compilar aquí, y en lugar de afirmar que funciona, se comprobó **lo que de verdad rompe un cliente móvil**: que los 17 campos del lote y los 7 de la página coincidan exactamente con lo que el servidor publica, nombre por nombre y nulabilidad por nulabilidad. Eso sí se puede saber sin un emulador, y es lo que más probabilidades tenía de estar mal.

Y el fallo de §10.1 —el formato de número que cambiaba con el idioma del teléfono— lo encontró una prueba del dominio antes de que existiera una pantalla donde verlo. Es el argumento entero a favor de haber dejado las reglas fuera de la capa de interfaz.